

Универзитет у Београду
Факултет организационих наука
Катедра за софтверско инжењерство

Студијски програм: Информациони системи и технологије Студијска
група: Софтверско инжењерство
Предмет: Програмирање репозиторијума података

п р п

ПРОГРАМИРАЊЕ РЕПОЗИТОРИЈУМА ПОДАТАКА

ПРОЈЕКАТ: Систем за праћење грешака у софтверу

Наставник:

Др Саша Д. Лазаревић, ред. проф.

Студент:

Алекса Живковић, 2022/0132

Асистенти:

Мср инж. Татјана Стојановић, асис.

Дипл. инж. Кристина Јовановић, сар.

Београд,
2026.

САДРЖАЈ

1. КОНЦЕПТУАЛИЗАЦИЈА 3

2. СПЕЦИФИКАЦИЈА 5

2.1. Спецификација решења проблема 5

2.1.1. ДЕР & РП 5

2.1.1.1. Дијаграм ентитета и релација 5

2.1.1.2. Речник податка 6

2.1.2. Структурна ограничења & СПИ 7

2.1.2.1. Структурна ограничења 7

2.1.2.2. Структурна правила интегритета (СПИ) 7

2.1.3. Вредносна ограничења & ВПИ 7

2.1.3.1. Вредносна ограничења 7

2.1.3.2. Вредносна правила интегритета (ВПИ) 8

2.2. Спецификација софтвера 9

2.2.1. Спецификација софтверске архитектуре 9

2.2.1.1. Спецификација архитектуре подсистема базе података 9

2.2.1.2. Спецификација архитектуре апликационог подсистема 10

2.2.2. Спецификација репозиторијума података 11

2.2.3. Спецификација апликација 11

2.2.4. Спецификација корисничког интерфејса 12

3. ИМПЛЕМЕНТАЦИЈА 12

3.1. Имплементација репозиторијума података 12

3.2. Имплементација апликација 13

3.3. Имплементација корисничког интерфејса 14

4. ЕКСПЛОАТАЦИЈА 14

5. РЕФЕРЕНЦЕ 15

6. ВИЗУЕЛНИ ЕЛЕМЕНТИ 16

ПОПИС СЛИКА

Слика 1 - ДЕР Систем за праћење грешака у софтверу 5

Слика 2 - Архитектура DB Solution-a 11

ПОПИС ТАБЕЛА

Табела 1 - Табела пројекат 4

Табела 2 - Табела грешка 4

Табела 3 - Табела коментар 4

Табела 6 - Дефиниција семантичких домена за ДЕР Систем за праћење грешака у софтверу 6

Табела 7 - Дефиниција атрибута за ДЕР Систем за праћење грешака у софтверу 6

Табела 8. Структурна правила интегритета за ДЕР Систем за праћење грешака у софтверу 7

Табела 9 - ВПИ за ограничења над доменом 9

Табела 10 - ВПИ за ограничења над вредностима атрибута 9



1. КОНЦЕПТУАЛИЗАЦИЈА

Функционални захтеви (ФЗ):

- 1) Неопходно је водити евиденцију о пројектима, грешкама и коментарима.
- 2) Атрибути пројекта: Ид, Naziv (јединствен), Opis, Verzija.
- 3) Атрибути грешке: Ид, Poruka, Ozbiljnost, DatumPrijave, StatusGr, IdProjekta.
- 4) Атрибути коментара: Ид, SadrzajXML, Autor, DatumKom, IdGreske.
- 5) Један пројекат може имати нула или више грешака; свака грешка припада тачно једном пројекту.
- 6) Једна грешка може имати нула или више коментара; сваки коментар припада тачно једној грешци.
- 7) Садржај коментара чува се у XML колони SadrzajXML и мора садржати елемент <tekst>.
- 8) Омогућити Full-Text претрагу по тексту грешке (Poruka) и опису пројекта (Opis) коришћењем CONTAINS/FREETEXT.
- 9) Омогућити CLR корисничку агрегатну функцију (UDA) са IBinarySerialize за прилагођене агрегате.
- 10) Омогућити напредну претрагу XML коментара коришћењем XQuery (FLWOR, функције, axis specifiers).

Увести ограничења:

- 1) Назив пројекта не сме бити празан и мора бити јединствен (UNIQUE).
- 2) Опис пројекта и текст грешке не смеју бити празни.
- 3) Озбиљност грешке мора бити вредност од 1 до 5 (1 = критична).
- 4) Статус грешке мора бити један од: „Отворена“, „УПроцесуРешавања“, „Решена“, „Затворена“.
- 5) Аутор коментара не сме бити празан.
- 6) SadrzajXML мора садржати елемент <tekst> (провера exist()).
- 7) Ид је идентификатор, неговорећа шифра, цео број већи од нуле.

Нефункционални захтеви (НЗ):

- 1) Базу података назвати [BugTracker]. База подржава колацију за српску ћирилицу (Serbian_Cyrillic_100_CI_AS).
- 2) Све што може декларативно – урадити декларативно, што не може тако – урадити употребом тригера, а што не може преко тригера – урадити преко ускладиштених процедура. Проверавају унете вредности и „избацивати“ кастомизоване изузетке.
- 3) За null колоне експлицитно навести да су null.
- 4) За приказе из базе података користити погледе (views).
- 5) Имплементирати базу као апстрактан тип података. Креирати слојевиту архитектуру SSS коришћењем схема: impl, spec и api_. Однос: impl → spec → api_.
- 6) Креирати апликациону улогу DataProvider* која користи објекте из схеме api_*, а нема права над схемама impl и spec.
- 7) Креирати две специјализоване API схеме:
 - (i) api_dev — за потребе апликације програмера (DEV);
 - (ii) api_qa — за потребе QA апликације.
- 8) Правила именовања: у api_* PascalCase процедуре / UPPER_CASE погледи / camelCase параметри; у spec upr_, fns_, fnt_, vw_; у impl tbl, vw, trg, idx.
- 9) Креирати слојевиту архитектуру апликације ApplicationDEV у C#. Приступ подацима искључиво преко улоге DataProviderDEV која користи api_dev.
- 10) Креирати слојевиту архитектуру апликације ApplicationQA у C#. Приступ подацима искључиво преко улоге DataProviderQA која користи api_qa.



Демо подаци:

Табела 1 - Табела пројекат

Id	Naziv	Opis	Verzija
1	WebShop	Онлајн продавница за малопродају електронике	2.4.1
2	MobileApp	Мобилна апликација за праћење поруџбина	1.8.3
3	ERPSystem	Систем за управљање пословним процесима	5.2.0

Табела 2 - Табела грешка

Id	IdProjekta	Poruka	Ozbiljnost	DatumPrijava	StatusGr
1	1	Корисник не може да се пријави након ресетовања лозинке	1	2025-03-10	Отворена
2	1	Слике производа се не учитавају на Safari прегледачу	3	2025-03-15	У Процесу Решавања
3	2	Апликација се руши при учитавању GPS локације	1	2025-04-01	Отворена
4	3	Извоз у PDF не ради за документе преко 100 страна	2	2025-02-20	Решена

Табела 3 - Табела коментар

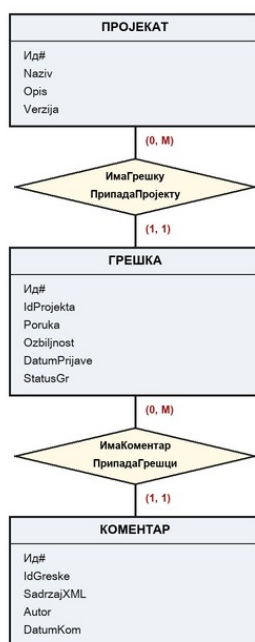
Id	IdGreske	SadrzajXML	Autor	DatumKom
1	1	<kom><tekst>Проблем у JWT токenu — истиче превремено</tekst><prioritet>Хитно</prioritet></kom>	Марко П.	2025-03-11 09:15:00
2	2	<kom><tekst>Репродуковано на Safari 17.3 и 17.4</tekst></kom>	Ана Н.	2025-03-16 14:00:00

2. СПЕЦИФИКАЦИЈА

2.1. Спецификација решења проблема

2.1.1. ДЕР & РП

2.1.1.1. Дијаграм ентитета и релација



Пословна правила: ГРЕШКА.ПрипадаПројекту.ДГ = 1 КОМЕНТАР.ПрипадаГрешци.ДГ = 1

Слика 1 - ДЕР Систем за праћење грешака у софтверу

АТРИБУТИ ЕНТИТЕТА:

ПРОЈЕКАТ: Ид#, Naziv, Opis (NVARCHAR MAX), Verzija;

ГРЕШКА: Ид#, IdProjekta, Poruka (NVARCHAR MAX), Ozbiljnost, DatumPrijave, StatusGr;

КОМЕНТАР: Ид#, IdGreske, SadržajXML (XML), Autor, DatumKom;

ОГРАНИЧЕЊА:

- (1) пословно правило: ГРЕШКА.PripadaProjektu.ДГ = 1
- (2) пословно правило: КОМЕНТАР.PripadaGresci.ДГ = 1

2.1.1.2. Речник податка

Табела 4 - Дефиниција семантичких домена за ДЕП Систем за праћење грешака у софтверу

Назив	Врста вредности	Темелјни домен	Ограничења			
			Формат	Обавезност	Подразумевана вредност	Вредност

» Нема посебно дефинисаних семантичких домена.

Табела 5 - Дефиниција атрибута за ДЕП Систем за праћење грешака у софтверу

Ентитет	Атрибут	Домен	Формат	Обавезност	Подразум. вредност	Ограничење вредности	Напомена
ПРОЈЕКАТ	Ид#	LongInt	/	not null	/	> 0	Идентификатор, IDENTITY
	Naziv	NVarChar(200)	/	not null	/	LEN > 0, UNIQUE	Јединствен назив
	Opis	NVarChar(MAX)	/	not null	/	LEN > 0	Full-Text индекс
	Verzija	NVarChar(20)	/	not null	/	LEN > 0	Верзија пројекта
ГРЕШКА	Ид#	LongInt	/	not null	/	> 0	Идентификатор, IDENTITY
	Poruka	NVarChar(MAX)	/	not null	/	LEN > 0	Full-Text индекс
	Ozbiljnost	Int	/	not null	/	BETWEEN 1 AND 5	1 = критична
	DatumPrijav	Date	/	not null	/	<= GETDATE()	Датум пријаве
	StatusGr	NVarChar(20)	/	not null	/	IN ('Отворена', 'УПроцесуРешавања', 'Решена', 'Затворена')	Енумерација
	IdProjekta	LongInt	/	not null	/	> 0	Спољни кључ → ПРОЈЕКАТ
КОМЕНТАР	Ид#	LongInt	/	not null	/	> 0	Идентификатор, IDENTITY
	SadrzajXML	XML	/	not null	/	Садржи <tekst>	CHECK преко exist()
	Autor	NVarChar(100)	/	not null	/	LEN > 0	Аутор коментара
	DatumKom	DateTime	/	not null	/	<= GETDATE()	Време коментара
	IdGreske	LongInt	/	not null	/	> 0	Спољни кључ → ГРЕШКА

2.1.2. Структурна ограничења & СПИ

2.1.2.1. Структурна ограничења

Структурна ограничења заснована на тривијалним пресликавањима:

» Нема структурних ограничења заснованих на тривијалним пресликавањима.

Структурна ограничења заснована на нетривијалним пресликавањима:

(1) ГРЕШКА.PripadaProjektu.ДГ = 1

(2) КОМЕНТАР.PripadaGresci.ДГ = 1

2.1.2.2. Структурна правила интегритета (СПИ)

Табела 6. Структурна правила интегритета за ДЕР Систем за праћење грешака у софтверу

Догађај		Ограничење	Акција	Порука	
Ентитет	Операција			Врста	Текст
ПРОЈЕКАТ	Insert	/	/	/	/
	Delete	(1)	Restrict ГРЕШКА	Info	Пројекат има евидентиране грешке. Брисање није могуће.
	Update	/	/	/	/
ГРЕШКА	Insert	(1)	Restrict ПРОЈЕКАТ	Info	Пројекат не постоји. Није могуће убацити ГРЕШКУ.
	Delete	(2)	Cascade КОМЕНТАР	Warn	Брисањем грешке бришу се и припадајући коментари.
	Update	/	/	/	/
КОМЕНТАР	Insert	(2)	Restrict ГРЕШКА	Info	Не постоји наведена грешка. Коментар не може бити унет.
	Delete	/	/	/	/
	Update	/	/	/	/

2.1.3. Вредносна ограничења & ВПИ

2.1.3.1. Вредносна ограничења

(А) Ограничења над вредностима једног атрибута

- Назив пројекта је јединствен и непразан; опис пројекта, текст грешке и аутор коментара нису празни.

```
CONSTRAINT ProjekatNaziv:=
    UNIQUE (Naziv) AND LEN(LTRIM(RTRIM(Naziv))) > 0
;
```

```
CONSTRAINT GreskaPoruka:=
    LEN(LTRIM(RTRIM(Poruka))) > 0
;
```

```
CONSTRAINT KomentarAutor:=
    LEN(LTRIM(RTRIM(Autor))) > 0;
```

2. Озбиљност грешке је од 1 до 5; статус из дозвољеног скупа.

```
CONSTRAINT GreskaOzbiljnost:=
    Ozbiljnost BETWEEN 1 AND 5
;
```

```
CONSTRAINT GreskaStatus:=
    StatusGr IN ('Отворена', 'УПроцесуРешавања', 'Решена', 'Затворена')
;
```

3. Садржај коментара (XML) мора садржати елемент <tekst>.

```
CONSTRAINT KomentarSadrzaj:=
    SadrzajXML.exist('//tekst') = 1
;
```

4. Ид је цео број већи од нуле.

```
CONSTRAINT ProjekatId / GreskaId / KomentarId:=
    Id > 0    yz    IDENTITY(1,1)
;
```

Сва ограничења су декларативна (CHECK / UNIQUE). Провера садржаја XML коментара реализује се CHECK ограничењем са exist() методом xml типа.

Опис пројекта има исто ограничење као текст грешке, дакле $LEN(LTRIM(RTRIM(Opis))) > 0$, а датуми не смеју бити у будућности: $DatumPrijave \leq CAST(GETDATE() AS DATE)$ и $DatumKom \leq GETDATE()$.

Ограничење KomentarSadrzaj не може стајати непосредно у CHECK клаузули. SQL Server на такав покушај враћа поруку Msg 423 и сам предлаже да се позив методе умота у скаларну функцију, па ограничење зове функцију impl.fnsImaTekst која враћа BIT.

(Б) Међузависност вредности два или више атрибута различитих ентитета

Не постоји изражена међузависност вредности атрибута различитих ентитета. Садржинска валидација XML коментара (обавезни <tekst>) односи се на један атрибут и обрађена је у тачки (А).

Аутоматско затварање статуса (нпр. trgAutoStatusResen) реализује се тригером, али не представља вредносно ограничење интегритета.

2.1.3.2. Вредносна правила интегритета (ВПИ)

Табела 7 - ВПИ за ограничења над доменом

Догађај			Ограничење	Акција	Порука	
Ентитет	Атрибут	Операција			Врста	Текст

Табела 8 - ВПИ за ограничења над вредностима атрибута

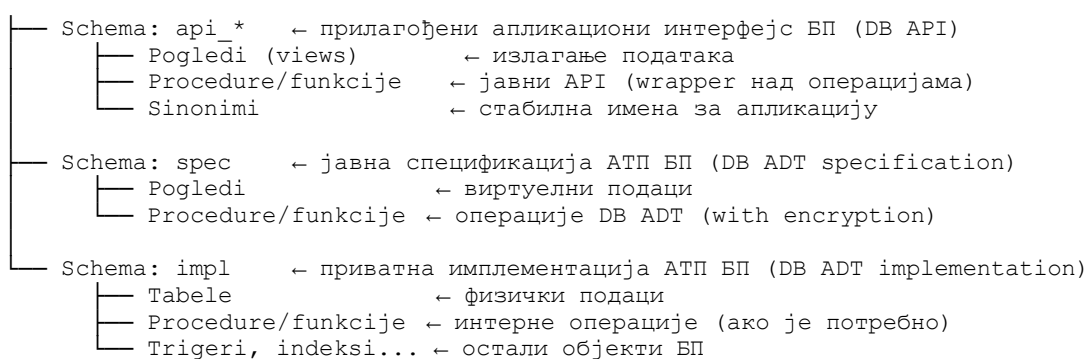
Догађај		Ограничење	Акција	Порука	
Ентитет	Операција			Врста	Текст
ПРОЈЕКАТ	Insert/Update PROJEKAT.Naziv	ProjekatNaziv	Reject	Warn	Назив пројекта мора бити јединствен и непразан.
ГРЕШКА	Insert/Update GRESKA.Ozbiljnost	GreskaOzbiljnost	Reject	Warn	Озбиљност мора бити вредност од 1 до 5.
	Insert/Update GRESKA.StatusGr	GreskaStatus	Reject	Warn	Статус мора бити у листи дозвољених вредности.
	Insert/Update GRESKA.Poruka	GreskaPoruka	Reject	Warn	Текст грешке не сме бити празан.
КОМЕНТАР	Insert KOMENTAR.SadrzajXML	KomentarSadrzaj	Reject	Warn	XML коментара мора садржати елемент <tekst>.
ПРОЈЕКАТ / ГРЕШКА / КОМЕНТАР	Insert/Delete *.Id	*Id	Recalculate	Non	/

2.2. Спецификација софтвера

2.2.1. Спецификација софтверске архитектуре

2.2.1.1. Спецификација архитектуре подсистема базе података

Подсистем базе података [BugTracker] (DB Solution [BugTracker])



Креирати слојевиту архитектуру SSS (SQL Server Solution-a), у овом случају названу DB Solution [EtrgovinaDB]:

DB Solution = DB API + DB ADT Specification + DB ADT Implementation
DB ADT = DB ADT Specification + DB ADT Implementation

коришћењем схема (schema):

- **api_*** – једна или више екстерних схема чине слој DB API; у овом слоју се креира

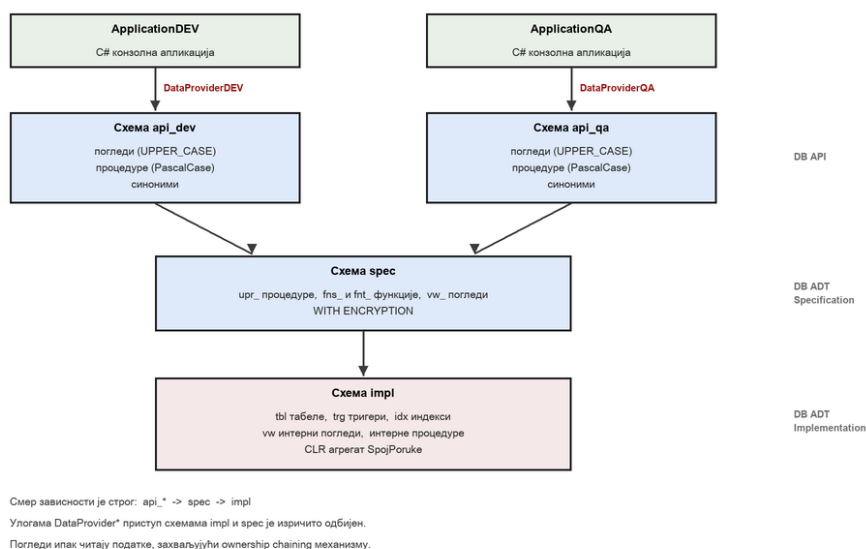
програмски интерфејс за приступ подацима у бази, који ће бити изложен апликационим програмерима; реализује се употребом операција АТП БП смештених у схеми спес;

- **спес** – спецификација АТП БП; изложени, тј. јавни део АТП БП; овде се креирају процедурални механизми за приступ подацима смештеним у табелама из impl; користи се одредба WITH ENCRYPTION да би се енкапулирала имплементација АТП;
- **impl** – имплементација АТП БП; енкапулирани, тј. приватни део АТП БП; креирање базе, логина, улога, корисника, табела, тригера, индекса, интерних процедура, функција и погледа и сл.

Две специјализоване API Schema-e:

- (1) **api_dev** — за потребе апликације програмера (DEV);
- (2) **api_qa** — за потребе QA апликације.

Апликационе улоге: DataProviderDEV (api_dev), DataProviderQA (api_qa).



Слика 2 - Архитектура DB Solution-a

Схеме DB API слоја на слици 2 су api_dev, за апликацију програмера, и api_qa, за QA апликацију.

2.2.1.2. Спецификација архитектуре апликационог подсистема

Обе апликације су конзолне, писане у језику C#, и обе су подељене на три слоја. Сваки слој је засебан фолдер у пројекту.

Домен (фолдер Model) садржи класе са подацима: Projekat, Greska, Komentar, Oznaka, Okruzenje, PromenaStatusa и остале. Тај слој не зна ни за базу ни за екран.

Слој приступа подацима (фолдер Podaci) садржи класу DataProviderDEV, односно DataProviderQA. То је једино место у апликацији које зна за објекат SqlConnection и једино које зна имена објеката у бази. Ништа не исписује на екран.

Слој приказа (фолдер Ui) садржи класе Meni и Ispis. Зна за домен и за слој испод, али не садржи ниједан ред SQL кода.

Улазна тачка Program.cs само саставља слојеве, повеже се на базу, активира апликациону улогу и покрене мени.

Апликација се повезује логином AppLoginDEV, односно AppLoginQA. Ти логини немају ниједно право над подацима, чак ни над сопственом API схемом. Права се добијају тек позивом

системске процедуре `sp_setapprole`, којом сесија губи свој идентитет и добија идентитет апликационе улоге. Пошто улога важи за сесију а не за поједину наредбу, апликација држи једну отворену везу током целог рада, а у везном низу је постављено `Pooling=False`, јер веза са активном апликационом улогом не може уредно да се врати у базен веза.

2.2.2. Спецификација репозиторијума података

Табеле са следећим колонама:

ПРОЈЕКАТ (Id, Naziv, Opis, Verzija)

ГРЕШКА (Id, Poruka, Ozbiljnost, DatumPrijave, StatusGr, IdProjekta)

КОМЕНТАР (Id, SadržajXML, Autor, DatumKom, IdGreske)

где је IdProjekta спољни кључ према табели ПРОЈЕКАТ, а IdGreske спољни кључ према табели ГРЕШКА. Колоне Opis и Poruka су под Full-Text индексом.

2.2.3. Спецификација апликација

Направљене су две апликације, обе са менијем, и обе раде искључиво преко своје API схеме. Свака ставка менија има тачно одређен објекат у бази иза себе.

ApplicationDEV, апликација програмера:

1. Пројекти 2. Грешке 3. Отворене грешке 4. Детаљи грешке 5. Пријави грешку 6. Додај коментар 7. Промени статус 8. Претрага 9. Нађи по термину 10. Активност аутора 11. Матрица грешака 12. Историја статуса.

ApplicationQA, апликација за контролу квалитета:

1. Преглед пројеката 2. Матрица грешака 3. Отворене грешке 4. Сажетак порука по пројекту, где ради CLR агрегат 5. Активност аутора 6. Извоз извештаја у XML фајл 7. Претрага 8. Нађи по термину 9. Грешке по оперативном систему 10. Историја статуса 11. Провера граница приступа.

QA апликација само чита. Ниједна њена ставка не мења податке, јер ни схема `api_qa` не излаже ниједну процедуру која мења.

Разлика двеју API схема није формална. Поглед `api_dev.KOMENTARI` има колону са сировим XML садржајем, која програмеру треба при дијагностици, док је `api_qa` верзија тог погледа нема. Схема `api_qa` уместо тога има збирне прегледе и извештаје.

2.2.4. Спецификација корисничког интерфејса

Кориснички интерфејс је конзолни, са нумерисаним менијем. Конзола је изабрана зато што је документација наводи као довољну и зато што не заклања оно што је предмет овог пројекта, а то је рад базе.

Ставка се бира уносом броја, нула значи излаз, а после сваког приказа се чека потврда да резултат не одјуре са екрана.

Табеле се исписују са поравнатим колонама чија се ширина рачуна према најдужој вредности, а предугачак текст се скраћује да ред не би прелазио у следећи.

Ћирилица се исписује исправно зато што се излаз и улаз конзоле при покретању постављају на UTF-8.

Списак статуса није прекуцан у апликацији него се чита из базе, преко синонима DOZVOLJENI_STATUSI. Ако се списак статуса прошири, апликација то види без измене кода.

Обавезна поља се траже, а необавезна се могу прескочити притиском на Enter, при чему се празно поље бази шаље као NULL а не као празан низ.

Поруке о грешци деле се на две врсте. Бројеви од 50000 навише су пословни изузеци из процедура, са поруком из табела СПИ и ВПИ, и они се приказују кориснику онакви какви јесу. Све испод тога је техничка грешка сервера и приказује се уз број грешке. На пример, ако корисник унесе озбиљност изван опсега, добија поруку: Одбијено: Озбиљност мора бити вредност од 1 до 5.

3. ИМПЛЕМЕНТАЦИЈА

3.1. Имплементација репозиторијума података

Рб.	Група	Опис захтева	SQL концепт	Референца
1	Обавезно	Креирати базу [BugTracker] са ћириличком колацијом. Креирати схеме impl, spec, api_dev, api_qa. Унети демо податке.	CREATE DATABASE, CREATE SCHEMA	—
2	Обавезно	Имплементирати табеле. CHECK: Ozbilnost BETWEEN 1 AND 5 и StatusGr у листи. CASCADE DELETE КОМЕНТАР при DELETE ГРЕШКА. DML тригер trgAutoStatusResen.	DDL, CHECK, CASCADE, DML тригер	—
3	Обавезно	Креирати stored процедуре у spec: upr_PrijaviGresku, upr_DodajKomentar, upr_PromeniStatus. TRY...CATCH са логовањем у tblErrorLog.	Stored процедуре, обрада грешака	—
4	Обавезно	Креирати погледе у spec: vw_GRESKA (са називом пројекта), vw_KOMENTAR, vw_OTVORENE_GRESKE. Wrapper у api_dev и api_qa.	CREATE VIEW, слојевита архитектура	—
5	Обавезно	Креирати апликационе улоге DataProviderDEV (пун приступ api_dev) и DataProviderQA (SELECT api_qa). Верификовати изолацију.	APPLICATION ROLE, GRANT, DENY	—

6	Обавезно	Креирати Full-Text Catalog и Full-Text Index над Poruka (tblGreska) и Opis (tblProjekat). Применити CONTAINS() са Булоским операторима (AND, OR, NOT) и NEAR.	Full-Text Search — CONTAINS, NEAR, Boolean	K4, стр. 273–274
7	Обавезно	Применити FREETEXT() и упоредити резултате са CONTAINS() за исте термине. Документовати разлике.	Full-Text Search — FREETEXT vs CONTAINS	K4, стр. 273
8	Обавезно	Имплементирати CLR UDA у C# са свим методама (Init, Accumulate, Merge, Terminate) и IBinarySerialize — UDA конкатенира поруке грешака.	CLR UDA + IBinarySerialize	K3, стр. 121–136; K4, стр. 193–200
9	Обавезно	Написати FLWOR XQuery над SadržajXML за груписање коментара по аутору и сортирање по датуму. Применити count().	XQuery FLWOR + count()	K1, стр. 311–320; K2, стр. 88–95
10	Обавезно	Применити axis specifiers (child::, descendant-or-self::, parent::) у XPath путањама унутар XQuery за навигацију кроз XML структуру коментара.	XQuery — Axis specifiers	K1, стр. 296–298
11	Обавезно	Написати stored процедуру која комбинује Full-Text CONTAINS() са xml nodes() за проналазак грешака чији коментари садрже задати термин.	Full-Text + xml nodes()	K4, стр. 273; K2, стр. 66–68
12	Оцена 10	Применити PIVOT за приказ броја грешака по пројекту у колонама по озбиљности (1–5). Процедура upr_MatricaGresaka у api_qa.	PIVOT — матрица грешака	K1, стр. 48–53; K4, стр. 66–73
13	Оцена 10	Применити OUTPUT клаузулу у UPDATE статуса грешке тако да се свака промена архивира у tblHistorijaStatusa у истом DML кораку.	OUTPUT клаузула — историја статуса	K1, стр. 39–41; K4, стр. 76–77

3.2. Имплементација апликација

Обе апликације су подељене на слојеве описане у одељку 2.2.1.2.

Активирање улоге. Процедура sp_setapprole важи за сесију а не за поједину наредбу, па апликација држи једну отворену везу током целог рада, уз Pooling=False у везном низу.

Параметризовани упити. Ниједна вредност се не лепи у текст упита него се шаље као параметар, чиме је онемогућено уметање страног SQL кода. Необавезни филтер је написан у облику услова који пропушта све када је параметар NULL.

Излазни параметри. Идентификатор нове грешке не враћа се наредбом SELECT него излазним параметром, дакле једним скаларом уместо целог резултујућег скупа.

Више резултујућих скупова. Процедура за детаље грешке враћа пет скупова редом, а апликација их чита методом NextResult, чиме се цео екран добија једним одласком до базе.

Обрада NULL вредности. Колоне које смеју бити празне читају се преко помоћне методе, јер би

позив GetString над празном вредношћу изазвао изузетак.

Склапање XML садржаја. Апликација шаље поља форме а документ склапа база, чиме је обавезни елемент гарантован, а специјални знакови исправно замењени.

3.3. Имплементација корисничког интерфејса

Класа Ispis садржи цртање табеле са поравнатим колонама, постављање питања и приказ порука, а класа Meni садржи редослед екрана и позива слој испод. Ниједан ред SQL кода не постоји у том слоју. Ћирилица се исписује исправно зато што се излаз и улаз конзоле постављају на UTF-8. Поруке о грешци деле се по броју: пословне изнад 50000 приказују се дословно, а техничке уз број грешке.

4. ЕКСПЛОАТАЦИЈА

Израдити тестове функционалних и нефункционалних захтева – написати упите којима се тестира урађено, изазивају грешке, итд. Креирати две C# апликације. Апликације могу бити једноставне, довољно је да буду конзолне и да се користе уз помоћ имплементираног менија, али можете направити апликацију по жељи (десктоп, веб...).

Студент предаје следеће ставке (можете направити јавни репозиторијум на GitHub-у):

1. SQL скрипту за креирање базе, схема, улога и корисника
2. SQL скрипту за креирање табела са свим ограничењима (impl слој)
3. SQL скрипту за унос демо података са верификационим тестовима
4. SQL скрипте за сваки захтев засебно (процедуре, функције, погледи, тригери)
5. C# assembly пројекат са CLR функцијама/процедурама/тригерима (ако постоји)
6. Кратак опис решења по захтеву (до пола стране) — шта је урађено и зашто
7. Тест SQL скрипте које демонстрирају рад сваког захтева са примерима из демо података
8. Апликације у прог. језику C#.

Одбрана пројектног задатка подразумева објашњење и тестирање свега што је урађено. За напредне задатке и задатке за оцену 10, пожељно је направити кратку презентацију да се теоријски објасне концепти који су рађени, а затим приказати и пример. Презентацију треба урадити по прослеђеном шаблону. Сама одбрана рада подразумева усмени део испита.

5. РЕФЕРЕНЦЕ

- Саша Д. Лазаревић, *Програмирање и подаци: Спецификација софтвера*, III свеска, ИИ ЛСИ, Београд: ФОН, 2025.
- Dušan Petković, *SQL Server 2019: A Beginner's Guide*, 7th ed., New York: McGraw-Hill Education, 2020.
- Саша Д. Лазаревић, *Алгебарска спецификација семантике апстрактних типова података*, ИИ ЛСИ, Београд: ФОН, 2015.
- Paul Nielsen, *SQL Server Bible*, Indianapolis: Wiley Publishing, 2007.
- С. Д. Лазаревић, „Релациони упитни језик“, у *Софтверско инжењерство - практикум за припремање пријемног испита за МАС СП Софтверско инжењерство*, В. Девеџић, С. Влајић, С. Д. Лазаревић, Уредници, Београд, Факултет организационих наука, 2017, pp. 276-347.
- Бранислав Лазаревић и др, *Базе података*, Београд: Факултет организационих наука, 2017.

(K1) [1] M. C. Coles, Ed., *Pro T-SQL 2005 Programmer's Guide*. in SpringerLink Bücher. Berkeley, CA: Apress, 2007. doi: 10.1007/978-1-4302-0387-2.

(K2) [2] S. Klein, *Professional SQL Server 2005 XML*. Indianapolis, Ind: Wiley, 2006.

(K3) [3] R. Dewson and J. Skinner, Eds., *Pro SQL Server 2005 Assemblies*. in SpringerLink Bücher. Berkeley, CA: Apress, 2006. doi: 10.1007/978-1-4302-0113-7.

(K4) [4] T. Rizzo et al., Eds., *Pro SQL Server 2005*. in SpringerLink Bücher. Berkeley, CA: Apress, 2006. doi: 10.1007/978-1-4302-0094-9.

6. ВИЗУЕЛНИ ЕЛЕМЕНТИ

Универзитет			
Факултет			
Катедра			КСИ
Лабораторија			ЛСИ
Предмет			